

POLITECNICO
MILANO 1863

054323 - INTERNET OF THINGS

POLITECNICO DI MILANO

COMPUTER SCIENCE AND ENGINEERING

Report: IoT Challenge 1

Authors:
Carmen Giaccotto (Person Code: ********)
Alessia Franchetti-Rosada (Person Code: ********)

Date: March 30, 2026

Contents

1 System design **3**
1.1 System overview . . . 3
1.2 Hardware configuration . . . 3
1.3 Software logic and ESP-NOW implementation . . . 3
1.4 Duty cycle and Deep Sleep . . . 4
1.5 Source code . . . 4

2 Power and energy consumption estimation **5**
2.1 Methodology . . . 5
2.2 Power and timing specifications . . . 5
2.2.1 Data extraction criteria . . . 5
2.2.2 Characterization of operational states . . . 6
2.3 Energy consumption per cycle . . . 7
2.4 Battery lifetime estimation . . . 7
2.5 Visual analysis . . . 8
2.6 Discussion of results . . . 9

3 Proposed improvements **9**
3.1 Analysis of the base system and proposed optimizations . . . 9
3.2 Optimized system implementation . . . 10
3.3 Recomputed energy consumption and comparison . . . 11
3.3.1 Scenario A: unchanged state cycle (no Wi-Fi) . . . 11
3.3.2 Scenario B: state changed cycle (Wi-Fi On at 2 dBm) . . . 11
3.3.3 New battery lifetime (software optimizations only) . . . 11
3.3.4 Discussion on energy plot and design choices . . . 12

1 System design

1.1 System overview

The primary objective of this project is to develop a commercial-style IoT motion sensor designed for a smart home environment. The sensing node is responsible for detecting human presence and measuring ambient illuminance levels. These environmental parameters are then transmitted to a central controller, referred to as the SINK node, which processes the data to make automated decisions such as determining whether the room lights should be switched on or off. To ensure long-term operation the device operates on a strict duty cycle, alternating between brief active sensing phases and prolonged periods of low-power operation.

1.2 Hardware configuration

The core of the hardware architecture is based on the ESP32 microcontroller, selected for its integrated Wi-Fi capabilities and native support for low-latency communication protocols. The environmental sensing is performed by two distinct components:

- **PIR (Passive Infrared) Sensor**: utilized to detect motion, connected to the digital pin 13. This pin was selected as it is a standard GPIO that does not interfere with the microcontroller's boot sequence.
- **LDR (Light Dependent Resistor)**: employed to measure ambient luminosity, connected to the analog pin 34. Pin 34 is specifically chosen because it interfaces directly with the ADC1 (Analog-to-Digital Converter 1) peripheral. On the ESP32 architecture, ADC2 is heavily shared with the Wi-Fi driver; therefore, using ADC1 ensures stable and accurate analog readings while the wireless module is active.

1.3 Software logic and ESP-NOW implementation

The software execution follows a sequential logic initialized upon every boot cycle. First, the system configures the designated pins and reads the current states of the PIR and LDR sensors. Once acquired, this data is formatted into a standardized string payload. Depending on the detected motion status, the payload strictly follows one of two specified formats:

1. **"MOTION_DETECTED-LUMINOSITY:ZZZ"**,
2. **"MOTION_NOT_ DETECTED-LUMINOSITY:ZZZ"**;

"ZZZ" represents the environmental illuminance converted into Lux, providing a reliable metric for automated lighting decisions.

For data transmission, the system utilizes the **ESP-NOW** protocol. The ESP32 is initialized in Wi-Fi Station mode (WIFI_STA) and the ESP-NOW interface is subsequently started. Because the central SINK node is assumed to possess no energy constraints and acts as a universal receiver in this emulation, the payload is transmitted using the universal broadcast MAC address (FF:FF:FF:FF:FF:FF).

1.4 Duty cycle and Deep Sleep

To heavily reduce energy consumption, the sensor node is programmed to enter a Deep Sleep state immediately after the ESP-NOW transmission is completed. During Deep Sleep, the main CPU and wireless peripherals are powered down, leaving only the Real-Time Clock (RTC) active to manage the wake-up timer. The duration of this sleep phase, denoted as X, is calculated based on Carmen Giaccotto's person code (********) by extracting the last two digits (AB = 37) and applying the required formula:

$$X = \frac{(37 \pmod{50}) + 5}{10} = 4.2 \text{ s} \tag{1}$$

The microcontroller converts this value into microseconds (4200000 μs) and passes it to the esp_sleep_enable_timer_wakeup() function. Once the timer expires, the node wakes up and resumes normal operation from the beginning of the setup phase.

1.5 Source code

During the development phase, two variations of the code were produced:

- An initial version was developed for instrumentation purposes. This version includes the injection of the micros() function at various stages of the execution. It was exclusively utilized within the Wokwi emulator to accurately measure the execution time of each operational state. The data gathered from this version was fundamental for the energy estimations discussed in Section 2.
- The definitive challenge code where all the extra measurement functions were removed to make the code faster and more efficient.

2 Power and energy consumption estimation

This section details the methodology and calculations used to estimate the energy consumption of the sensor node and determine its expected battery life.

2.1 Methodology

The energy estimation follows a hybrid approach combining empirical measurements from simulation and real-world power profiles:

- **Execution Times (t):** measured using the *micros()* function within the Wokwi emulator. To account for simulation variability, 20 complete operational cycles were recorded at 100% CPU speed and arithmetic means were calculated for each operational phase.
- **Average Power (P):** extracted from the provided CSV datasets. Milliwatt (mW) values were averaged across representative time windows for each state and converted to Watts (W) for energy calculations.
- **Initial Parameters:** derived from Carmen Giaccotto's person code (********).
  - **Deep Sleep Time (X):** computed from the last two digits ($AB = 37$), as shown in (1).
  - **Battery Energy (Y):** computed from the last four digits ($ABCD = 9837$).

$$Y = (9837 \pmod{5000}) + 15000 = 19837 \text{ J} \tag{2}$$

2.2 Power and timing specifications

2.2.1 Data extraction criteria

To ensure the accuracy of the energy model, power parameters were extracted from the CSV datasets using specific statistical criteria.

- **Steady-State windowing:** for states characterized by a plateau (Deep Sleep and Sensor Reading), the average power was not calculated over the entire file, but rather on a "*Window of Interest*" (WoI) where the signal reached a steady state. This allowed for the exclusion of transient spikes or startup noise that do not represent the actual operational consumption.
- **Peak-Plateau averaging:** for the Wi-Fi Setup and Transmission phases, which exhibit high variability, the power was extracted by averaging the samples within the highest stable plateau. For P_{tx}, specifically, the criteria involved identifying the pulse corresponding to the ESP-NOW dispatch in the *sender.csv* trace and computing the arithmetic mean of the samples within that pulse duration.

2.2.2 Characterization of operational states

Applying the criteria above, the power values for the five distinct states were defined as follows:

- **Sensor Reading ($P_{sensing}$):** derived from `sensor-read.csv` during active CPU processing without radio activity ($\sim$ 350 mW).
- **Wi-Fi Setup (P_{wifi}):** averaged from the high-consumption plateau in `deep_sleep.csv` occurring after wake-up ($\sim$ 627 mW).
- **Transmission (P_{tx}):** extracted from the peak values in `sender.csv` during ESP-NOW data dispatch ($\sim$ 671 mW).
- **Idle/Logic (P_{idle}):** baseline active consumption during string formatting and logic ($\sim$ 247 mW).
- **Deep Sleep (P_{sleep}):** averaged from the steady-state period in `deep_sleep.csv` ($\sim$ 45.6 mW).

The residual time between the total active time and the sum of identified tasks (Sensing, WiFi, TX) is categorized as **Idle/Logic**, representing CPU overhead for data formatting and internal state management.

Operational State	**Mean Duration (t) [s]**	**Avg Power (P) [W]**	**Data Source**
Sensor Reading	0.0010455	0.350	*sensor-read.csv*
Wi-Fi Setup	0.1960990	0.627	*deep_sleep.csv*
Transmission (TX)	0.0000698	0.671	*sender.csv*
Idle / Logic	0.0006890	0.247	*Calculated (residual)*
Deep Sleep	4.2000000	0.0456	*deep_sleep.csv*
Total Cycle	**4.3979033**	-	-

Table 1: Average power and duration (mean of 20 cycles)

2.3 Energy consumption per cycle

Using the fundamental relation $E = P \cdot t$, we calculate the energy contribution for each phase.

- $E_{sensing} = 0.350 \cdot 0.0010455 \approx 0.000366$ J
- $E_{wifi} = 0.627 \cdot 0.196099 \approx 0.122954$ J
- $E_{tx} = 0.671 \times 0.0000698 \approx 0.000047$ J
- $E_{idle} = 0.247 \times 0.000689 \approx 0.000170$ J
- $E_{sleep} = 0.0456 \cdot 4.2 = 0.19152$ J

The total energy consumed per cycle is:

$$E_{cycle} = \sum E_{phases} \approx \mathbf{0.315057}\ \text{J} \tag{3}$$

2.4 Battery lifetime estimation

Given the total battery energy $Y = 19837$ J, the number of operating cycles is:

$$N_{cycles} = \frac{19837\ \text{J}}{0.315057\ \text{J/cycle}} \approx \mathbf{62963}\ \text{cycles} \tag{4}$$

The total lifetime in seconds is 62963×4.3979033 s ≈ 276905 seconds.
Converting to days:

$$\text{Lifetime (days)} = \frac{276905\ \text{s}}{86400\ \text{s/day}} \approx \mathbf{3.205}\ \text{days} \tag{5}$$

2.5 Visual analysis

The following figures provide a visual breakdown of the power and energy profile.

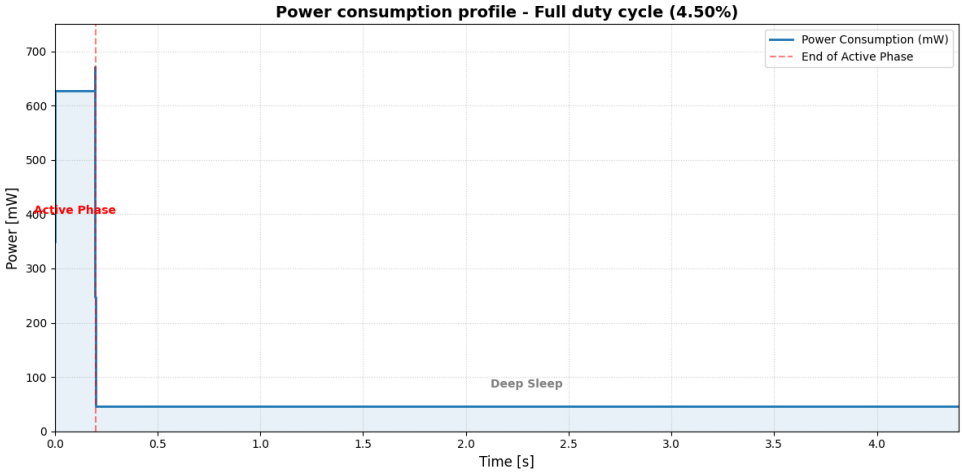


Figure 1: Power consumption profile over a full operational cycle. This visualization emphasizes the intermittent nature of the sensor node's activity. The active phase (marked in red) constitutes only approximately 4.5% of the total cycle, illustrating the efficiency of the Duty Cycle approach where the device spends the vast majority of its lifetime in a low-power Deep Sleep state to conserve energy.

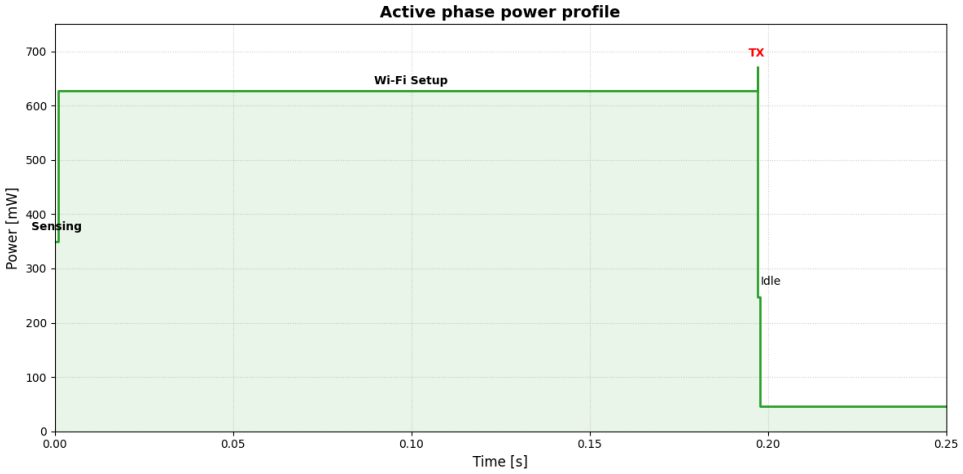


Figure 2: Detailed view of the power profile during the active phase (first 250 ms). This granular perspective highlights the different sub-states of the microcontroller: from the initial sensor sampling to the energy-intensive radio stack initialization (Wi-Fi Setup) and the final ESP-NOW data transmission (TX). This decomposition is fundamental for identifying the radio initialization as the primary time bottleneck during active operation.

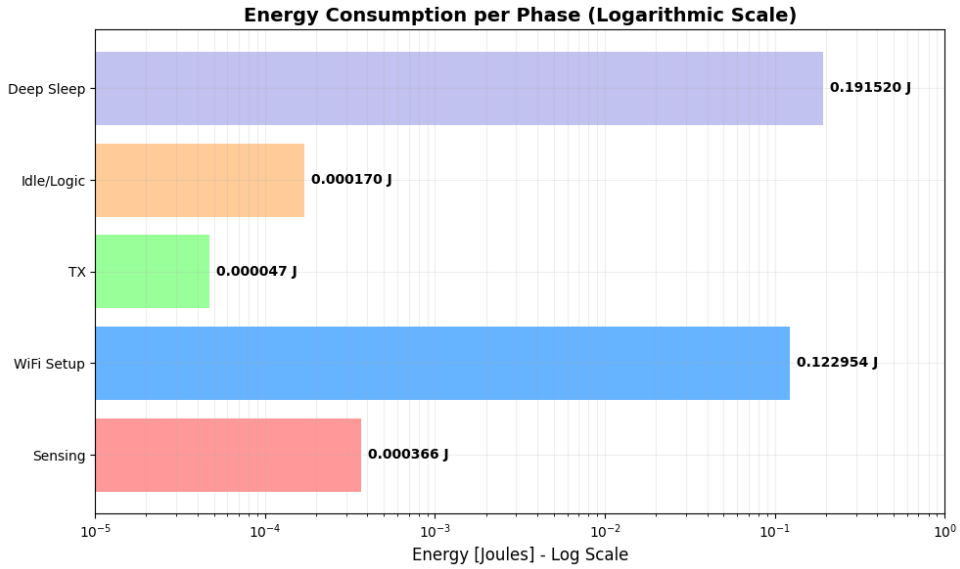


Figure 3: Due to the massive difference in magnitude between sleep and active tasks, a logarithmic scale is used to visualize all phases. While Sensing and TX have high power, their energy impact is minimal due to their extremely short duration compared to Deep Sleep.

2.6 Discussion of results

The estimated lifetime of approximately 3.2 days is significantly constrained by the high energy consumption during the Deep Sleep state, which accounts for over 60% of the total energy per cycle ($\sim$ 191 mJ out of 315 mJ). This high baseline is typical of commercial ESP32 development boards, where peripheral circuitry continues to draw current even when the SoC is in sleep mode. This highlights a clear opportunity for the optimizations discussed in the following sections.

3 Proposed improvements

3.1 Analysis of the base system and proposed optimizations

Starting from the energy estimation of the base system, two critical bottlenecks were identified:

- **Wi-Fi activation overhead:** turning on the radio module and initializing ESP-NOW in every single cycle consumes a significant amount of energy ($E_{wifi} \approx 0.122$ J), accounting for nearly 40% of the active cycle energy.
- **Deep Sleep baseline power:** the measured Deep Sleep power (45.6 mW) is exceptionally high for a microcontroller. This indicates the use of an ESP32 develop-

ment board (which includes an always-on power LED, a USB-to-UART bridge and an inefficient voltage regulator) rather than a barebone ESP32 chip.

To drastically reduce energy consumption while maintaining the system's core objective (notifying the sink node to support lighting decisions) the following optimizations are proposed:

- **Improvement 1 - Event-Driven transmission (software):** instead of transmitting the sensor state blindly every 4.2 seconds, the node should only activate its Wi-Fi module when a change in state occurs (e.g., from "no motion" to "motion detected" and vice versa). If the room is empty and the light level hasn't changed significantly, the sensor wakes up, checks the sensors, notices no change and immediately goes back to sleep. This avoids the massive Wi-Fi energy penalty. To implement this, we can use the ESP32's `RTC_DATA_ATTR` memory, which retains both the previous motion state and the last measured Lux value even during Deep Sleep.
- **Improvement 2 - TX power reduction (software):** as hinted by the provided sender.csv profile, the default transmission power (19.5 dBm) is unnecessarily high for an indoor smart home scenario. By reducing the TX power to 2 dBm, we can lower the peak transmission power from ∼671 mW to ∼621 mW.
- **Improvement 3 - Hardware selection (hardware design):** for a real world commercial deployment, the ESP32 DevKit must be replaced with a barebone ESP32-WROOM module or a modern ESP32-C3. This hardware shift reduces Deep Sleep consumption from 45.6 mW to approximately 10 μW.

3.2 Optimized system implementation

To translate these proposed optimizations into a functional prototype, the project was revised. The optimized Arduino sketch introduces an event-driven logic by utilizing the `RTC_DATA_ATTR` macro. This allows the `lastMotionState` variable to survive the Deep Sleep reset. Upon waking up, the node reads both sensors; if the motion state is identical to the previous cycle and the Lux variation is negligible (e.g., less than 50 Lux), the firmware bypasses the entire Wi-Fi initialization process and immediately re-enters Deep Sleep, saving substantial energy. Furthermore, when a state change does occur and transmission is necessary, the instruction `WiFi.setTxPower(WIFI_POWER_2dBm)` is executed right after enabling the Wi-Fi module, restricting the transmission power to 2 dBm. The hardware topology remains unchanged.

Note on implementation and simulation: the optimized code utilizes the `RTC_DATA_ATTR` macro to retain the previous sensor state in the RTC memory during Deep Sleep, allowing the event-driven logic to function correctly on physical ESP32 hardware. However, due to known limitations within the Wokwi emulator regarding Deep Sleep and RTC

memory retention (where variables are often reset upon wake-up), the simulated serial output might continuously transmit. The provided `.ino` code reflects the architecturally correct implementation for a real-world WSN node.

3.3 Recomputed energy consumption and comparison

To validate the optimizations, we recompute the energy parameters.

3.3.1 Scenario A: unchanged state cycle (no Wi-Fi)

If the room state is unchanged (which happens most of the day), the ESP32 only powers on, reads the sensor and sleeps. The execution time drops drastically (approx. 2 ms active time).

$$E_{active_notx} = P_{sensing} \cdot t_{sensing} + P_{idle} \cdot t_{idle} = (0.350 \cdot 0.0010455) + (0.247 \cdot 0.000689) \approx 0.000536 \text{ J}$$
$$E_{cycle_notx} = E_{active_notx} + E_{sleep} = 0.000536 \text{ J} + 0.19152 \text{ J} = 0.192056 \text{ J}$$

3.3.2 Scenario B: state changed cycle (Wi-Fi On at 2 dBm)

When motion starts or stops, the device transmits. Using 2 dBm lowers the TX power to 0.621 W.

$$\begin{aligned} E_{tx_new} &= 0.621 \cdot 0.0000698 = 0.000043 \text{ J} \\ E_{cycle_tx} &= E_{sensing} + E_{wifi} + E_{tx_new} + E_{idle} + E_{sleep} \\ &= 0.000366 + 0.122954 + 0.000043 + 0.000170 + 0.19152 = 0.315053 \text{ J} \end{aligned}$$

Assuming an average smart home scenario where the room state changes only 5% of the time (e.g., people entering/leaving a few times a day), the new average energy per cycle is:

$$E_{avg} = (0.95 \cdot 0.192056 \text{ J}) + (0.05 \cdot 0.315053 \text{ J}) = 0.198206 \text{ J / cycle}$$

3.3.3 New battery lifetime (software optimizations only)

$$\text{Cycles} = \frac{19837 \text{ J}}{0.198206 \text{ J}} = 100083 \text{ cycles}$$
$$\text{Duration} = 100083 \cdot 4.3970 \text{ s} = 440155 \text{ s} \approx \mathbf{5.08 \text{ days}}$$

(A ~60% increase in battery life compared to the original 3.2 days).

3.3.4 Discussion on energy plot and design choices

If we plot the power-versus-time graph for the optimized system, the visual difference is striking. The original plot showed high-power Wi-Fi peaks (over 600 mW) occurring exactly every 4.2 seconds. The new optimized plot is predominantly flat at the Deep Sleep baseline, interrupted only by tiny 350 mW spikes for sensor readings. The massive 600 mW transmission peaks only appear sporadically, aligning exclusively with physical events in the real world.

While the software optimizations increased the lifespan to 5 days, the true practical impact is made by the hardware choice. If Improvement 3 (using a barebone ESP32) is applied, E_{sleep} drops to ~0.000042 J. The average cycle energy would go to ~0.006 J, extending the battery life from 5 days to over 160 days on the exact same battery. This demonstrates that while algorithmic efficiency is crucial in WSNs, choosing the right hardware is what makes an IoT device last long enough for the real world.

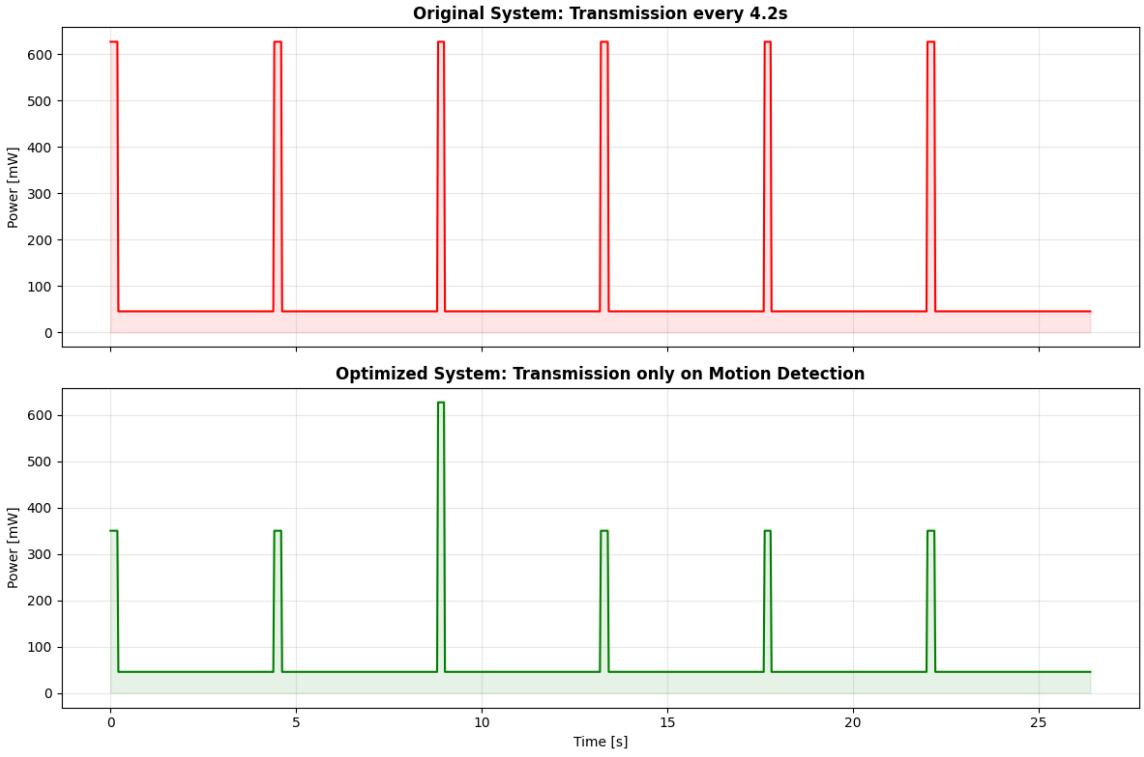


Figure 4: Comparison between the original power profile (top) and the optimized one (bottom).

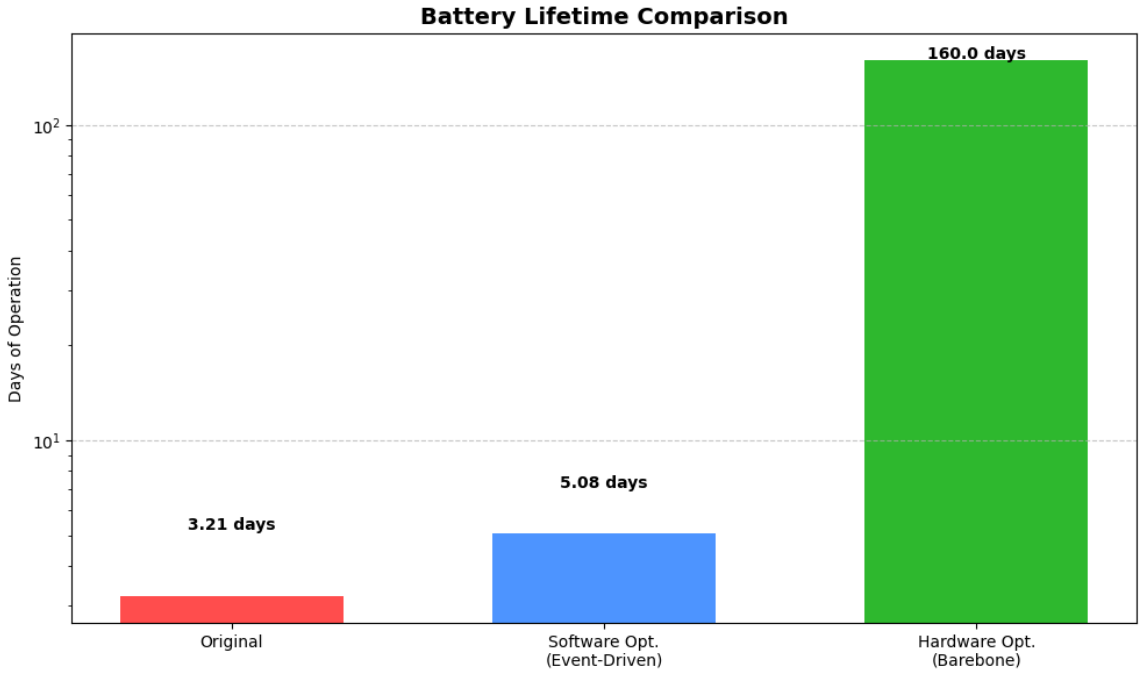


Figure 5: Battery lifetime projection across different design stages. The chart illustrates the incremental gains from software optimizations and the exponential improvement provided by hardware-appropriate selection (barebone ESP32).

Students' contributions

Carmen and Alessia worked together to design the Wokwi circuit, write the ESP32 code and implement the ESP-NOW communication protocol. Carmen focused on extracting the execution times from the simulation and calculating the energy consumption profile. Alessia was primarily responsible for writing the optimization proposal. Both students collaborated on solving the theoretical exercise, writing, reviewing and formatting the final report.